Microservices Security Challenges

- The usage of many loosely coupled services connected into one application is a different app development approach compared to the monolithic architecture.
- The foremost microservices security challenges are mainly caused by the following.
  - Decoupled Architecture
    Every microservice uses a dedicated database and uses diverse technologies. Moreover, the developers need to create a more complex solution to enable microservices authentication and authorization for every service.
  - Microservices Communication
    Many decoupled services with dedicated functionality need to communicate and share sensitive information.

Microservices Security Implementation - Best Practices

Microservices Security Implementation . . . cont'd

Authentication and Authorization

- A new authentication and authorization solution is needed since the microservices security architecture comprises many interconnected services.
- It should be different from solutions used in the monolithic architecture because every service should verify a user ID and fetch additional user info.

Monolith vs microservices authentication / authorization approach

	Monolithic Architecture	Microservices Architecture
Login	Usage of one database	Dedicated login service is required to run authentication & authorization
Identify Token Verification	One backend app issues and verifies ID tokens	ID tokens are issued & verified by different services
Additional USer Info Access	All the data is stored in same database	Data & user info are stored in different databases

Microservices authentication / authorization implementation options

To implement secure authentication and authorization patterns in microservices architecture, following options are available.

- All-in-one authentication service — The authentication service verifies users' identities and creates a session, issuing an identity token. Every service verifies tokens and gets additional user info separately.
- Shared session storage — The authentication service checks users' identity and creates sessions that are stored in dedicated shared storage. Services verify user ID and get additional user info by connecting to the shared session storage.
- JSON[1] web tokens (JWT) — The authentication service checks the user identity and issues a JWT token. The JWT token is provided to services, which verify it and get basic user info.

[1] See end slides for JSON details

Microservices authentication / authorization impl. options . . . cont'd

	Pros	Cons	Key facts
All-in-One Auth Service	Great encapsulation Basic user info is always up-to-date.	High load on authentication service; Resource services strictly depend on authentication service.	The best fit for microservices Performance tradeoff should be considered.
Shared Session Storage	Easy to implement No strict dependencies between authentication & resource services.	Poor encapsulation Authentication service may become bottleneck Basic user info is not always up-to-date.	Great performance with tradeoff of some microservices principles.
JSON Web Tokens (JWT)	No performance bottlenecks or points of failure; Promotes the decentralization approach Basic user info can be read from a JWT.	More data to transfer over network JWT digital signature should be validated; Difficult to implement Revoke JWT problem.	A comprehensive option, but difficulties in handling service logouts. Senior developers should implement it.

Microservices authentication / authorization impl. options . . . cont'd

	All-in-One Auth service	Shared Session Storage	JSON Web Tokens (JWT)
Login	Distinctive login API	Distinctive login API, but sessions are created in a shared storage	Authentication service creates a JWT containing user info
Identity Token Verification	Authentication service is called every time on need	Services connect the shared session storage instead of the authentication service	Services verify received tokens and get user info from them
Additional User Info Access	Usage of a dedicated authentication service API	Microservice authentication service API is used to fetch additional user info	Additional info can be stored in a JWT or fetched using the authentication service API upon request

Ready-to-use customer identity and access management solutions

Ready-to-use Customer Identity and Access Management (CIAM) systems can help speed up the development and address most security concerns by opting for a well-tested solution.

Three Widely used CIAM platforms are listed below:

- AWS Cognito
- FusionAuth
- Auth0

Ready-to-use customer identity & access management solutions . . . cont'd

CIAM Platform	Key Features
AWS Cognito	Self-registration, Customizable UI, Multi-factor authentication, Migration options, Compromised credential protection, identity store, microservices security risks assessment, Access to AWS resources, machine-to-machine authentication
FusionAuth	Single sign-on, multifactor authentication, biometric authentication, social & gaming logins, breached pwd detection, IoT authorization
Auth0	Social login, Single sign-on, Multifactor authentication, real-time breaches detection, serverless developer tools, passwordless authentication

Microservices Security Implementation . . . cont'd

- Secured Communication Between Microservices
  - Microservices need to communicate with each other to send/receive data constantly.
  - Secured communication between services is key in enabling security in microservices.
- Best practices for securing microservices communication
  - Use HTTPS
    - Hypertext transfer protocol secure (HTTPS) is a secure protocol that enables services to communicate with each other using end-to-end encryption.
    - It's advisable to configure every service to validate certificates of services they need to connect automatically.
  - Use a service mesh

    The term "service mesh" defines an extra layer in a network that adds more features and improves the microservices security. A mesh network implies sidecars enabled for services, which work as proxies. The usage of a mesh network in microservices foresees the opportunity for developers to:
    - simplify communication between services
    - detect communication errors and failures
    - encrypt and validate data
    - streamline the development, testing, and deployment of applications